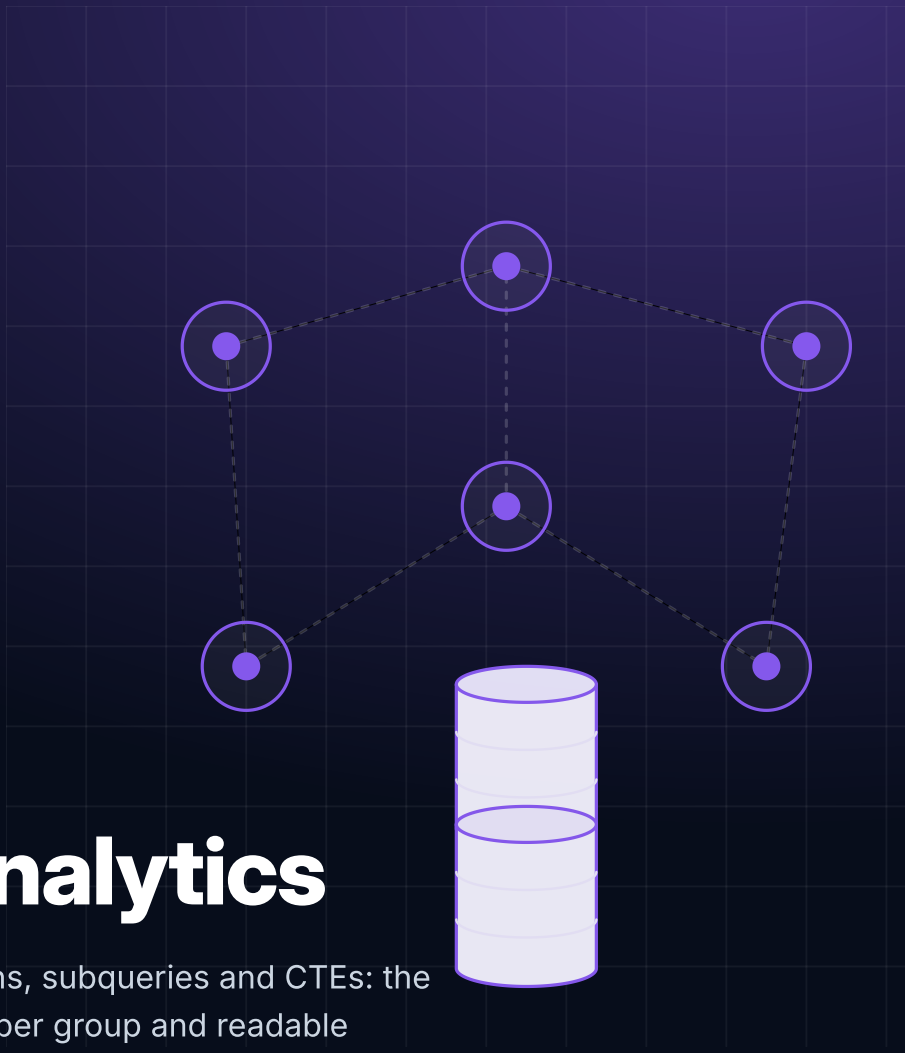


PHASE 06 / 07

Advanced Analytics

Window functions, ranking functions, subqueries and CTEs: the tools behind running totals, Top-N per group and readable multi-step queries.

Window Functions | Frames | Ranking | Subqueries | CTEs

This is where SQL stops being 'a filter with totals' and becomes an analytics language. Almost every data-engineering interview and every real warehouse model uses the patterns in this phase.

CONTENTS

Inside this phase

Window Functions 3
- Collapse versus keep 3
- The OVER clause: anatomy 4
- A first window: the running total 4
- PARTITION BY versus GROUP BY 5
- Naming a window once: the WINDOW clause 7

Frames & Value Functions 8
- Frame boundaries 8
- ROWS vs RANGE: what happens with ties 9
- The course's frame examples 10
- Moving averages 10
- Value functions: first, last, nth 11
- LAG and LEAD: peek at neighbors 12

Ranking Functions 13
- The course's slide: 90, 90, 100 13
- Ranking inside groups 14
- NTILE: equal-sized buckets 15
- PERCENT_RANK and CUME_DIST 16
- Top-N per group: the pattern to memorize 16

Subqueries 18
- Where a subquery can live 18
- 1 · Scalar subquery in WHERE (the course example) 19
- 2 · Column subquery with IN 20
- 3 · Derived table in FROM (course example 2) 20
- 4 · Correlated subqueries 21
- 5 · EXISTS and NOT EXISTS 22

Common Table Expressions (CTEs) 24
- Subquery → CTE 24
- Reusing a CTE twice 25
- Recursive CTEs 26
- CTE, subquery, temp table or view? 28

Phase 6 Recap & Practice 29
- Cheat sheet 29
- Check yourself 29
- Practice lab 30
- Answer key 31

6.1 PART 6.1 · ANALYTICS WITHOUT COLLAPSING ROWS

Window Functions

An aggregate such as `SUM` squeezes many rows into one. A **window function** does the same arithmetic but *keeps every row*, adding the result as a new column. That single idea unlocks running totals, rankings, comparisons with the previous row and much more.

What you will learn

- Explain how a window function differs from `GROUP BY`
- Read and write the `OVER ( PARTITION BY … ORDER BY … )` clause
- Build a running total with `SUM() OVER (ORDER BY …)`
- Reuse one window definition with the `WINDOW` clause

Why it matters for data engineering. Reporting questions are rarely 'one number per group'. They are 'each order *and* its share of the customer total', 'each month *and* the change from last month'. Window functions answer them in a single pass, with no self-joins.

Collapse versus keep

GROUP BY (aggregate)

item	category	price
A	Toys	10
B	Toys	30
C	Books	20
D	Books	40
E	Books	60

SUM →

category	SUM(price)
Toys	40
Books	120

5 rows in → 2 rows out
the individual items are gone

OVER () (window)

item	category	price
A	Toys	10
B	Toys	30
C	Books	20
D	Books	40
E	Books	60

SUM →

item	cat_total
A	40
B	40
C	120
D	120
E	120

5 rows in → 5 rows out
every row keeps its identity and gains a group-level number

Rule of thumb: GROUP BY changes the grain of the result. OVER() never does.

Figure 1. Same SUM, two behaviors. The window version can show the item and its group total side by side.

The OVER clause: anatomy

A window function is any function followed by `OVER (…)`. The parentheses describe *which rows the function may look at* for each output row.

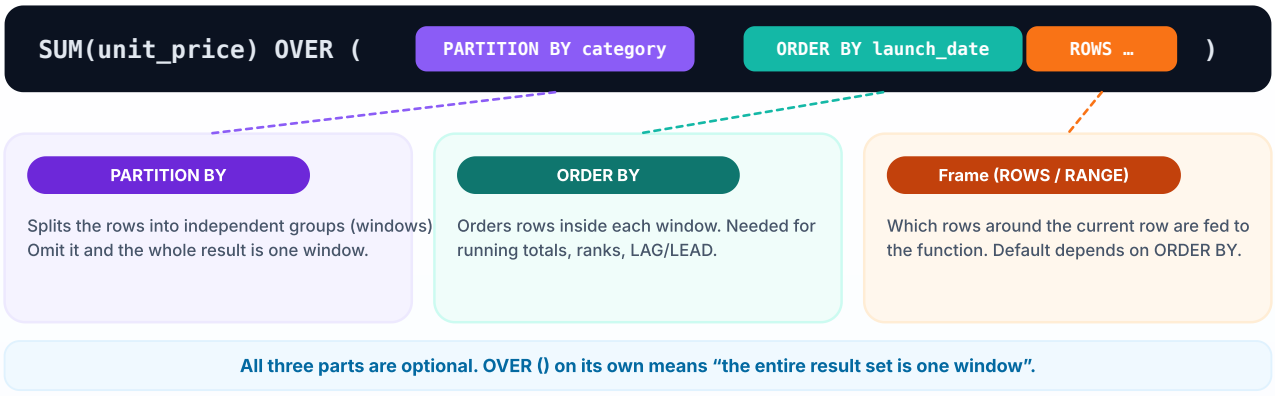


Figure 2. Anatomy of a window function call. The colored parts below explain each clause.

A first window: the running total

This is the first example from the course: add `SUM(unit_price) OVER (ORDER BY launch_date)` to the product table. Every product stays visible, and the new column accumulates the price in launch order.

SQL — Each row shows its own price and the total so far. Notice there is no GROUP BY.

```sql
SELECT   product_name, launch_date, unit_price,
         SUM(unit_price) OVER (ORDER BY launch_date) AS running_total
FROM     dim_product
ORDER BY launch_date
LIMIT    6;
```

RESULT — 6 ROWS

product_name	launch_date	unit_price	running_total
Good Both	2020-10-18	382.05	382.05
Exist Himself	2020-10-20	189.22	571.27
Special Stage	2020-10-22	201.47	772.74
Investment Listen	2020-10-26	250.31	1023.05
Half Garden	2020-10-27	968.37	1991.42
Message Artist	2020-10-28	213.28	2204.70

TWO BEHAVIORS OF ONE FUNCTION

With `ORDER BY` inside `OVER`, an aggregate becomes **cumulative**. Without it (`SUM(x) OVER ()`) every row gets the same grand total.

SQL — Share-of-total: the row's price divided by its category's total, all in one pass

```sql
SELECT category, product_name, unit_price,
       SUM(unit_price) OVER ()                                  AS grand_total,
       SUM(unit_price) OVER (PARTITION BY category)             AS category_total,
       ROUND(100 * unit_price / SUM(unit_price) OVER (PARTITION BY category), 2) AS pct_of_category
FROM   dim_product
ORDER  BY category, product_name
LIMIT  5;
```

RESULT — 5 ROWS

category	product_name	unit_price	grand_total	category_total	pct_of_category
Books	Actually Southern	16.75	247895.03	41314.05	0.04
Books	After Week	527.00	247895.03	41314.05	1.28
Books	Although Claim	288.31	247895.03	41314.05	0.70
Books	Although Cultural	411.13	247895.03	41314.05	1.00
Books	Among Manager	82.40	247895.03	41314.05	0.20

DATA-ENGINEERING LENS

Percent-of-total and running totals are everywhere in dashboards. Before window functions this took a self-join or a correlated subquery (slow). A window function computes it in a single sort, which is why modern warehouse SQL (Snowflake, BigQuery, Redshift, Databricks) leans on it heavily.

PARTITION BY versus GROUP BY

	GROUP BY	PARTITION BY (window)
Rows returned	One per group	Every input row
Detail columns allowed?	No (only keys and aggregates)	Yes, select anything
Where it lives	A clause of the query	Inside `OVER (…)` of each function
Different groupings in one query?	No, one `GROUP BY`	Yes, each function can partition differently
Filter on the result	`HAVING`	Wrap in a subquery/CTE, then `WHERE` (see 6.3)

THE CLASSIC WINDOW-FUNCTION ERROR

You cannot use a window function in `WHERE`. It is calculated *after* `WHERE`, `GROUP BY` and `HAVING` have run. To filter on its value, compute it in a subquery or CTE and filter outside. The execution-order diagram below shows why.

SQL — Window function in WHERE is rejected

```sql
SELECT product_name, ROW_NUMBER() OVER (ORDER BY unit_price) AS rn FROM dim_product WHERE
ROW_NUMBER() OVER (ORDER BY unit_price) <= 3;
```

ERROR

```
ERROR 3593 (HY000): You cannot use the window function 'row_number' in this context.
```

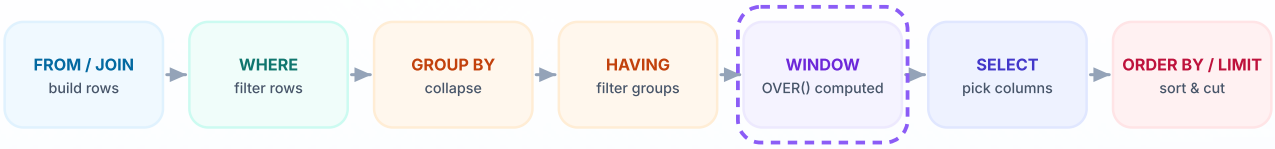


Figure 3. Logical execution order (from Phase 3), now with the window stage. Filter on window results one level up.

SQL — Fix: compute inside a derived table, filter outside

```sql
SELECT * FROM (
    SELECT product_name, unit_price, ROW_NUMBER() OVER (ORDER BY unit_price) AS rn
    FROM   dim_product
) t
WHERE rn <= 3;
```

RESULT — 3 ROWS

product_name	unit_price	rn
Middle Forward	11.82	1
Look Interest	12.15	2
Environment Performance	12.22	3

Naming a window once: the WINDOW clause

When several functions share the same `OVER (…)`, define it once and refer to it by name. It keeps long queries readable and prevents typos.

SQL — One window `w`, three functions. (`monthly_rev` is the monthly revenue we built from the fact table.)

```sql
SELECT ym, revenue,
       SUM(revenue)        OVER w AS running_revenue,
       ROUND(AVG(revenue)  OVER w) AS running_avg,
       COUNT(*)            OVER w AS months_so_far
FROM   monthly_rev
WINDOW w AS (ORDER BY ym)
LIMIT  5;
```

RESULT — 5 ROWS

ym	revenue	running_revenue	running_avg	months_so_far
2023-10	319207	319207	319207	1
2023-11	619450	938657	469329	2
2023-12	620670	1559327	519776	3
2024-01	690291	2249618	562405	4
2024-02	618678	2868296	573659	5

6.2 PART 6.2 · WHICH ROWS DOES THE WINDOW SEE?

Frames & Value Functions

`PARTITION BY` picks the group and `ORDER BY` sorts it. The **frame** then answers a finer question: for *this* row, which neighbors are included? Changing the frame turns the same `SUM` into a running total, a moving average or a whole-group total.

What you will learn

- Write frames with `ROWS BETWEEN … AND …`
- Choose between `ROWS` and `RANGE` and understand how ties behave
- Compute moving averages and cumulative sums
- Use `FIRST_VALUE`, `LAST_VALUE`, `NTH_VALUE`, `LAG` and `LEAD`

Why it matters for data engineering. Moving averages smooth noisy metrics, `LAG` gives period-over-period change, and `FIRST_VALUE` / `LAST_VALUE` pick a group's first and last events. All of them are frame-based, and the `LAST_VALUE` default-frame trap is a famous interview question.

Frame boundaries

Keyword	Meaning
`UNBOUNDED PRECEDING`	from the first row of the partition
`n PRECEDING`	n rows before the current row
`CURRENT ROW`	the row being calculated
`n FOLLOWING`	n rows after the current row
`UNBOUNDED FOLLOWING`	up to the last row of the partition



Figure 4. The same current row (position 5) with four different frames. Shaded cells are the rows the function can see.

ROWS vs RANGE: what happens with ties

`ROWS` counts physical rows. `RANGE` works on *values*: all rows whose `ORDER BY` value ties with the current row are **peers** and enter the frame together. The default frame, when you write `ORDER BY` but no frame, is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.

day	amount	default (RANGE)	ROWS
Jan 1	100	100	100
Jan 2	200	350	300
Jan 2	50	350	350
Jan 3	150	500	500
Jan 4	300	800	800

Two rows share Jan 2

RANGE treats them as peers: both see the Jan 2 total (350). ROWS goes strictly row by row: 300, then 350.

Tip: add a unique tiebreaker to ORDER BY, or use ROWS.

Figure 5. Running total over `ORDER BY day` with a duplicated date. The default frame groups the two Jan 2 rows.

SQL — Real output: the Jan 2 rows both show 350 under the default frame, but 300 then 350 under ROWS

```sql
SELECT day, amount,
       SUM(amount) OVER (ORDER BY day)                          AS default_range,
       SUM(amount) OVER (ORDER BY day ROWS UNBOUNDED PRECEDING)  AS rows_frame
FROM   daily;
```

RESULT — 5 ROWS

day	amount	default_range	rows_frame
2025-01-01	100	100	100
2025-01-02	200	350	300
2025-01-02	50	350	350
2025-01-03	150	500	500
2025-01-04	300	800	800

INTERVIEW ANGLE

"Why does my running total show the same value on two rows?" Because the default frame is `RANGE`, and tied `ORDER BY` values are peers. Fix: `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, or add a unique column to the `ORDER BY`.

The course's frame examples

SQL `CURRENT ROW` ends the frame at me (running total). `UNBOUNDED FOLLOWING` extends it to the end, so every row sees the same total.

```sql
SELECT   product_name, launch_date, unit_price,
         SUM(unit_price) OVER (ORDER BY launch_date
             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)    AS cumulative,
         SUM(unit_price) OVER (ORDER BY launch_date
             ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS whole_table
FROM     dim_product
ORDER BY launch_date
LIMIT    4;
```

RESULT 4 ROWS

product_name	launch_date	unit_price	cumulative	whole_table
Good Both	2020-10-18	382.05	382.05	247895.03
Exist Himself	2020-10-20	189.22	571.27	247895.03
Special Stage	2020-10-22	201.47	772.74	247895.03
Investment Listen	2020-10-26	250.31	1023.05	247895.03

Moving averages

A moving average smooths short-term noise so the trend shows. Averaging the current month and the two before it is the frame `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`.

SQL The first rows have fewer than 3 months available, so `months_in_frame` is 1 and 2 there. Windows do not pad with zeros.

```sql
SELECT ym, revenue,
       ROUND(AVG(revenue) OVER w3) AS moving_avg_3m,
       COUNT(*)           OVER w3  AS months_in_frame
FROM   monthly_rev
WINDOW w3 AS (ORDER BY ym ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
LIMIT  6;
```

RESULT 6 ROWS

ym	revenue	moving_avg_3m	months_in_frame
2023-10	319207	319207	1
2023-11	619450	469329	2
2023-12	620670	519776	3
2024-01	690291	643470	3
2024-02	618678	643213	3
2024-03	590172	633047	3

PRO TIP

A **7-day moving average** over daily data is `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW`. Count the current row too, so 6 + 1 = 7.

Value functions: first, last, nth

Function	Returns	Frame-sensitive?
`FIRST_VALUE(x)`	x from the first row of the frame	yes
`LAST_VALUE(x)`	x from the last row of the frame	**yes, the trap**
`NTH_VALUE(x, n)`	x from the n-th row of the frame (NULL if frame is shorter)	yes
`LAG(x, n, default)`	x from n rows **before** the current row	no, ignores the frame
`LEAD(x, n, default)`	x from n rows **after** the current row	no, ignores the frame

SQL — `last_default` simply repeats the current row, because the default frame ends at CURRENT ROW. `last_real` needs the full frame.

```sql
SELECT ym, revenue,
       FIRST_VALUE(revenue) OVER w      AS first_month,
       LAST_VALUE(revenue)  OVER w      AS last_default,
       LAST_VALUE(revenue)  OVER w_all  AS last_real
FROM   monthly_rev
WINDOW w     AS (ORDER BY ym),
       w_all AS (ORDER BY ym ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
LIMIT  4;
```

RESULT — 4 ROWS

ym	revenue	first_month	last_default	last_real
2023-10	319207	319207	319207	200506
2023-11	619450	319207	619450	200506
2023-12	620670	319207	620670	200506
2024-01	690291	319207	690291	200506

WATCH OUT

`LAST_VALUE` **surprise.** With the default frame the 'last' row is always the current row, so the function looks broken. Always pair it with `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`. (`FIRST_VALUE` works by default, because the frame always starts at the beginning.)

LAG and LEAD: peek at neighbors

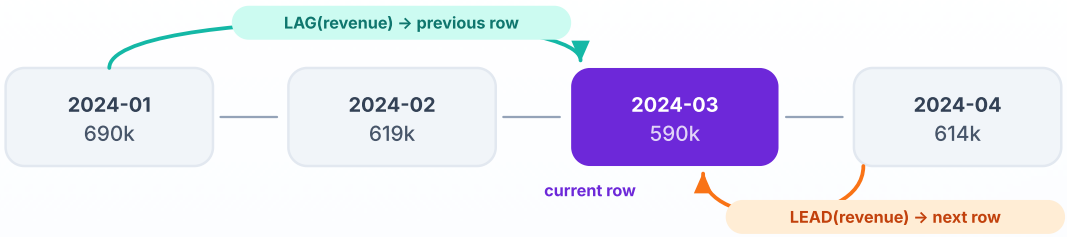


Figure 6. From the highlighted row, LAG reads one row back and LEAD one row forward, regardless of any frame.

SQL — Month-over-month growth. The first row has no previous month, so LAG gives NULL, and so does the percentage.

```sql
SELECT ym, revenue,
       LAG(revenue)  OVER (ORDER BY ym) AS prev_month,
       LEAD(revenue) OVER (ORDER BY ym) AS next_month,
       ROUND(100 * (revenue - LAG(revenue) OVER (ORDER BY ym)) / LAG(revenue) OVER (ORDER BY ym), 1)
AS mom_pct
FROM   monthly_rev
LIMIT  5;
```

RESULT — 5 ROWS

ym	revenue	prev_month	next_month	mom_pct
2023-10	319207	*NULL*	619450	*NULL*
2023-11	619450	319207	620670	94.1
2023-12	620670	619450	690291	0.2
2024-01	690291	620670	618678	11.2
2024-02	618678	690291	590172	-10.4

PRO TIP

`LAG(x, 1, 0)` supplies a **default** (0) when there is no previous row: `LAG(expr, offset, default)`. Phase 7 uses this exact form on the weather table.

6.3 PART 6.3 · NUMBERING ROWS INSIDE A WINDOW

Ranking Functions

Ranking functions assign a position to every row based on the `ORDER BY` of the window. They differ in only one thing: **what they do with ties**. Understanding that difference is half of the Top-N interview questions.

What you will learn

- Tell `ROW_NUMBER`, `RANK` and `DENSE_RANK` apart on tied values
- Rank within groups using `PARTITION BY`
- Split rows into buckets with `NTILE`
- Use `PERCENT_RANK` and `CUME_DIST` for relative position

Why it matters for data engineering. Top-N per group, 'second highest salary', de-duplication and percentile bucketing all start with a ranking function. Choosing the wrong one silently returns the wrong rows when ties exist.

The course's slide: 90, 90, 100

SQL — Two equal prices create a tie. Look at what each function does next.

```sql
SELECT unit_price, ROW_NUMBER() OVER (ORDER BY unit_price) AS `row_number`, RANK() OVER (ORDER BY unit_price) AS `rank`, DENSE_RANK() OVER (ORDER BY unit_price) AS `dense_rank` FROM slide;
```

RESULT — 3 ROWS

unit_price	row_number	rank	dense_rank
90	1	1	1
90	2	1	1
100	3	3	2

score	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
90	2	2	2
90	3	2	2
80	4	4	3
70	5	5	4

ROW_NUMBER
Always unique: 1, 2, 3, 4, 5.
Ties are broken arbitrarily.

RANK
Ties share a rank, then the next rank skips: 1, 2, 2, 4, 5.

DENSE_RANK
Ties share a rank, no gaps: 1, 2, 2, 3, 4.

Figure 7. Five exam scores with a tie at 90 (Ben and Cy). The orange cells show where RANK and DENSE_RANK diverge.

SQL — Real output that matches the diagram

```sql
SELECT student, score,
       ROW_NUMBER() OVER (ORDER BY score DESC) AS rn,
       RANK()       OVER (ORDER BY score DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY score DESC) AS dense
FROM   exam;
```

RESULT — 5 ROWS

student	score	rn	rnk	dense
Ana	100	1	1	1
Ben	90	2	2	2
Cy	90	3	2	2
Dia	80	4	4	3
Eli	70	5	5	4

REMEMBER

Which one do I need? *Exactly N rows, one per position* → `ROW_NUMBER`. *Olympic medals (1, 2, 2, 4)* → `RANK`. *'Third highest distinct value'* → `DENSE_RANK`.

Ranking inside groups

Add `PARTITION BY` and the numbering restarts for each group. Here is the course's second ranking example, using the most expensive products per category.

SQL — Numbering restarts at 1 for every category. Prices here are unique, so all three agree.

```sql
SELECT category, product_name, unit_price,
       ROW_NUMBER() OVER (PARTITION BY category ORDER BY unit_price DESC) AS rn,
       RANK()       OVER (PARTITION BY category ORDER BY unit_price DESC) AS rnk,
       DENSE_RANK() OVER (PARTITION BY category ORDER BY unit_price DESC) AS dense
FROM   dim_product
ORDER  BY category, rn
LIMIT  6;
```

RESULT — 6 ROWS

category	product_name	unit_price	rn	rnk	dense
Books	Only Sense	978.88	1	1	1
Books	Us Record	975.35	2	2	2
Books	Way Glass	962.74	3	3	3
Books	Claim Authority	941.43	4	4	4
Books	Less Other	940.38	5	5	5
Books	Investment Economic	922.31	6	6	6

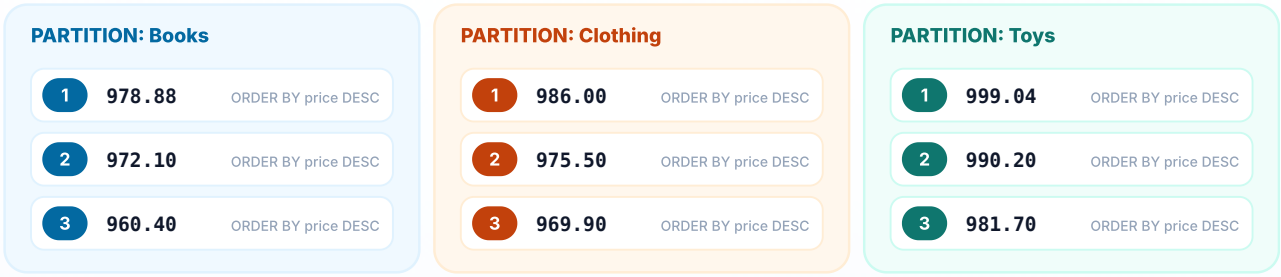


Figure 8. PARTITION BY restarts the counter. Each category is ranked as if the others did not exist. (Illustrative top-3 per partition.)

NTILE: equal-sized buckets

`NTILE(n)` splits the ordered rows into *n* buckets of (nearly) equal size and returns the bucket number. Use it for quartiles, deciles and 'top 10 %' segments. When rows do not divide evenly, the first buckets get one extra row.

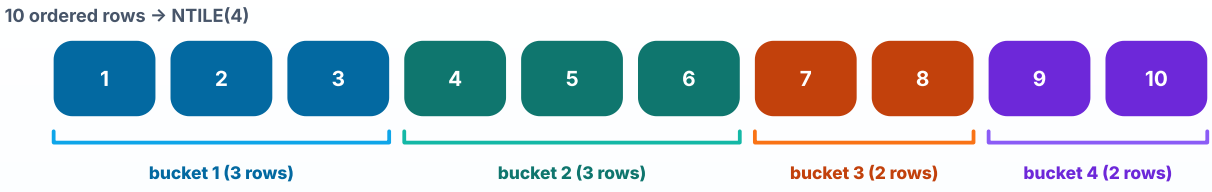


Figure 9. Ten rows into four buckets: 3 + 3 + 2 + 2. The remainder goes to the earliest buckets.

SQL — The most expensive Toys land in quartile 4

```sql
SELECT category, product_name, unit_price,
       NTILE(4) OVER (PARTITION BY category ORDER BY unit_price) AS price_quartile
FROM   dim_product
WHERE  category = 'Toys'
ORDER  BY unit_price DESC
LIMIT  4;
```

RESULT — 4 ROWS

category	product_name	unit_price	price_quartile
Toys	Likely Brother	999.04	4
Toys	Air War	967.14	4
Toys	Listen Discover	952.58	4
Toys	Shake True	932.05	4

PERCENT_RANK and CUME_DIST

Function	Formula	Range	Reads as
`PERCENT_RANK()`	(rank − 1) / (rows − 1)	0 to 1	share of rows **strictly below** me
`CUME_DIST()`	rows ≤ me / total rows	above 0 to 1	share of rows **at or below** me

SQL The lowest score has `PERCENT_RANK` 0. The highest has `CUME_DIST` 1. Ben and Cy tie, so they share both values.

```sql
SELECT student, score,
       PERCENT_RANK() OVER (ORDER BY score) AS pct_rank,
       CUME_DIST()    OVER (ORDER BY score) AS cume_dist
FROM   exam;
```

RESULT 5 ROWS

student	score	pct_rank	cume_dist
Eli	70	0.0000000000	0.2000000000
Dia	80	0.2500000000	0.4000000000
Ben	90	0.5000000000	0.8000000000
Cy	90	0.5000000000	0.8000000000
Ana	100	1.0000000000	1.0000000000

Top-N per group: the pattern to memorize

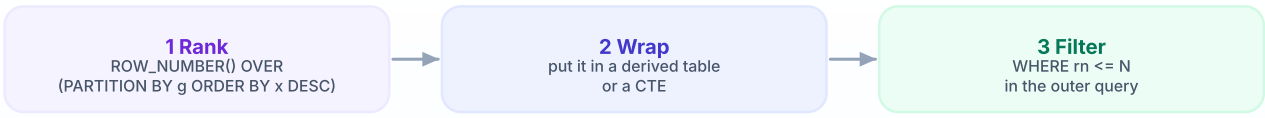


Figure 10. Window functions cannot be filtered in the same SELECT, so every Top-N query has these three moves.

SQL — The two most expensive products in each of the six categories

```sql
SELECT category, product_name, unit_price, rn
FROM (
    SELECT category, product_name, unit_price,
           ROW_NUMBER() OVER (PARTITION BY category ORDER BY unit_price DESC) AS rn
    FROM   dim_product
) ranked
WHERE rn <= 2
ORDER BY category, rn;
```

RESULT — 12 ROWS

category	product_name	unit_price	rn
Books	Only Sense	978.88	1
Books	Us Record	975.35	2
Clothing	Room Leg	986.00	1
Clothing	Religious Others	978.30	2
Electronics	Soon During	989.41	1
Electronics	Answer Hot	985.18	2
Home & Kitchen	Nature Along	980.27	1
Home & Kitchen	Program Parent	972.53	2
Sports	Car Report	986.45	1
Sports	Thing Probably	966.16	2
Toys	Likely Brother	999.04	1
Toys	Air War	967.14	2

INTERVIEW ANGLE

"Find the 5th highest price per category." Use `DENSE_RANK() … ORDER BY unit_price DESC`, then filter `= 5`. This is Scenario 1 of the course and appears in Phase 7.

6.4 PART 6.4 · A QUERY INSIDE A QUERY

Subqueries

A **subquery** is a `SELECT` written inside another statement. The inner query produces a value, a list or a whole table, and the outer query uses it. Whenever you catch yourself thinking *'first I need the average, then compare to it'*, you want a subquery.

What you will learn

- Recognize scalar, column, row and table (derived) subqueries
- Place a subquery in `WHERE`, `SELECT` or `FROM`
- Explain how a correlated subquery differs, and when to prefer `EXISTS`
- Avoid the `NOT IN` + NULL trap

Why it matters for data engineering. Subqueries let a single statement answer multi-step questions. They are also the stepping stone to CTEs (6.5), which express the same logic more readably, and they are a favorite interview topic (`IN` vs `EXISTS`, correlated vs not).

Where a subquery can live

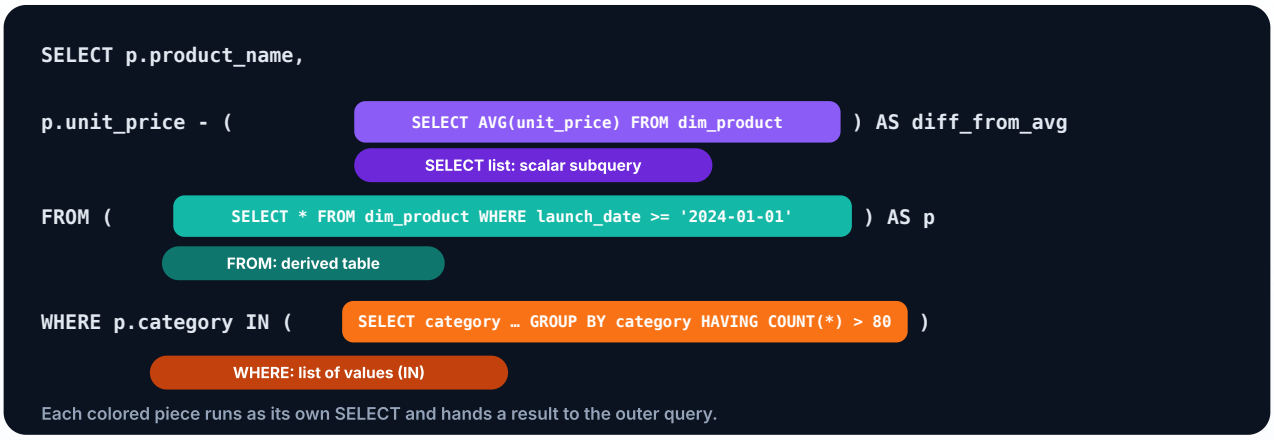


Figure 11. Three places a subquery can appear, and what each must return.

Kind	Returns	Used with	Example
Scalar	1 row × 1 column	`=` `<` `>` or in `SELECT`	`> (SELECT AVG(price) …)`
Column / list	many rows × 1 column	`IN`, `ANY`, `ALL`	`IN (SELECT customer_key …)`
Row	1 row × many columns	row constructor	`(a, b) = (SELECT x, y …)`
Table (derived)	many rows × many columns	`FROM (…) AS alias`	`FROM (SELECT … ) AS t`

1 · Scalar subquery in WHERE (the course example)

SQL — Step 1 by hand: the average price

```sql
SELECT AVG(unit_price) AS avg_price FROM dim_product;
```

RESULT — 1 ROW

avg_price
495.790060

SQL — Step 2: let the database do both steps. 250 of the 500 products are above average; here are the cheapest four of those.

```sql
SELECT product_name, category, unit_price
FROM   dim_product
WHERE  unit_price > (SELECT AVG(unit_price) FROM dim_product)
ORDER  BY unit_price
LIMIT  4;
```

RESULT — 4 ROWS

product_name	category	unit_price
Probably Month	Electronics	499.16
Tell Recognize	Books	499.97
Building Choose	Toys	503.80
His Serve	Books	505.44

SQL — A scalar position needs exactly one row

```sql
SELECT product_name FROM dim_product WHERE unit_price > (SELECT unit_price FROM dim_product WHERE
category = 'Toys');
```

ERROR

```
ERROR 1242 (21000): Subquery returns more than 1 row
```

PRO TIP

If the inner query may return several rows, switch operator: `IN (…)` for membership, `> ALL (…)` for 'greater than every', `> ANY (…)` for 'greater than at least one'.

2 · Column subquery with IN

SQL — The priciest product per category, using a row constructor `(a, b) IN (...)`

```sql
SELECT product_name, category, unit_price
FROM   dim_product
WHERE  (category, unit_price) IN (
           SELECT category, MAX(unit_price) FROM dim_product GROUP BY category)
ORDER  BY category;
```

RESULT — 6 ROWS

product_name	category	unit_price
Only Sense	Books	978.88
Room Leg	Clothing	986.00
Soon During	Electronics	989.41
Nature Along	Home & Kitchen	980.27
Car Report	Sports	986.45
Likely Brother	Toys	999.04

3 · Derived table in FROM (course example 2)

A subquery in `FROM` behaves as a temporary table. **MySQL insists it have an alias**, even if you never use it again.

SQL — Forgetting the alias

```sql
SELECT * FROM (SELECT * FROM dim_product WHERE unit_price > 900);
```

ERROR

```
ERROR 1248 (42000): Every derived table must have its own alias
```

SQL — A subquery nested inside a subquery: the filter runs on the derived table's 250 rows

```sql
SELECT *
FROM (
    SELECT *
    FROM   dim_product
    WHERE  unit_price > (SELECT AVG(unit_price) FROM dim_product)
) AS subquery_table
WHERE product_name = 'Figure Method';
```

RESULT — 1 ROW

product_key	product_id	product_name	category	brand	unit_price	launch_date
1	PROD0001	Figure Method	Clothing	BrandC	897.35	2020-12-06

> **NOTE**
> The nesting reads **inside-out**: the innermost `AVG` runs first, then the derived table, then the outer `WHERE`. That is exactly the readability problem CTEs (next section) were invented to fix.

4 · Correlated subqueries

So far the inner query ran *once*, independent of the outer one. A **correlated** subquery refers to a column of the outer row, so conceptually it re-runs *for every outer row*.

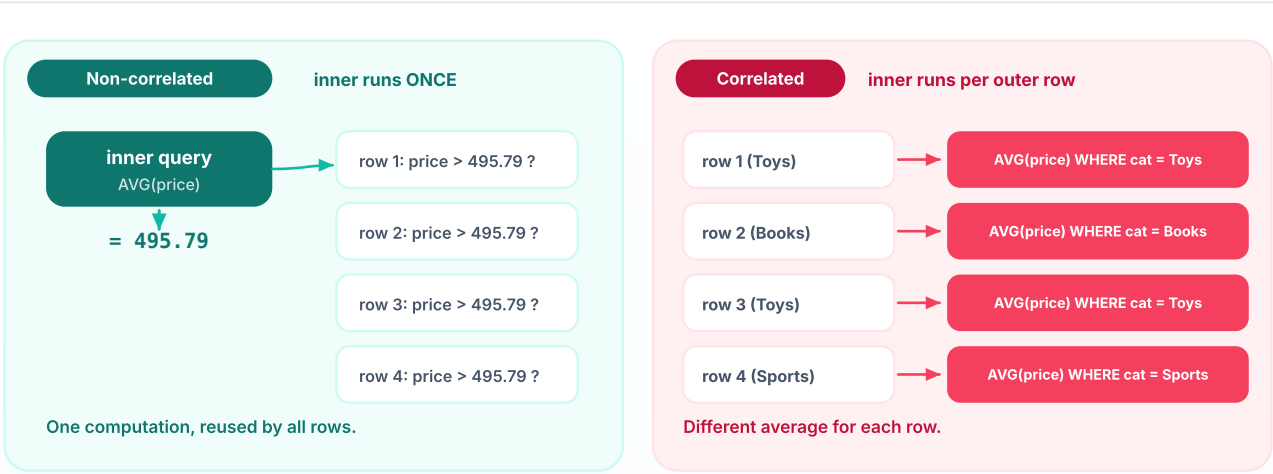


Figure 12. Left: the inner result is the same for every row. Right: the inner query depends on the current outer row.

SQL — Products priced above their own category's average (255 in total). `p2` is the inner alias; `p` is the outer row.

```sql
SELECT category, product_name, unit_price
FROM   dim_product p
WHERE  unit_price > (SELECT AVG(unit_price) FROM dim_product p2 WHERE p2.category = p.category)
ORDER  BY category, unit_price DESC
LIMIT  4;
```

RESULT — 4 ROWS

category	product_name	unit_price
Books	Only Sense	978.88
Books	Us Record	975.35
Books	Way Glass	962.74
Books	Claim Authority	941.43

The window version of the same question needs one pass and often reads better:

SQL — Same rows, same order. (Window beats correlated subquery for 'compare to group average'.)

```sql
SELECT category, product_name, unit_price
FROM (
    SELECT *, AVG(unit_price) OVER (PARTITION BY category) AS cat_avg
    FROM   dim_product
) t
WHERE unit_price > cat_avg
ORDER BY category, unit_price DESC
LIMIT 4;
```

RESULT — 4 ROWS

category	product_name	unit_price
Books	Only Sense	978.88
Books	Us Record	975.35
Books	Way Glass	962.74
Books	Claim Authority	941.43

5 · EXISTS and NOT EXISTS

`EXISTS (subquery)` is simply TRUE if the inner query finds **at least one row**. It never cares *what* the row contains, which is why people write `SELECT 1`. It stops at the first match, so it is the natural way to ask 'is there any related row?'.

SQL — Reminder of the tiny ecom tables: customer 301 has no orders

```sql
SELECT * FROM customers; SELECT * FROM orders;
```

RESULT — 3 ROWS

o_id	cust_id	price
1	101	1000
2	201	1100
3	501	1200

SQL — EXISTS and NOT EXISTS split the customers into two groups (NOT EXISTS is the anti-join of Phase 4)

```sql
SELECT c.name, 'has orders' AS status
FROM   customers c
WHERE  EXISTS (SELECT 1 FROM orders o WHERE o.cust_id = c.id)
UNION ALL
SELECT c.name, 'no orders'
FROM   customers c
WHERE  NOT EXISTS (SELECT 1 FROM orders o WHERE o.cust_id = c.id);
```

RESULT — 3 ROWS

name	status
love	has orders
ansh	has orders
lamba	no orders

The NOT IN trap

SQL — a = {1, 2, 3}, b = {1, NULL}. Two should be missing from b, but NOT IN finds none.

```sql
CREATE TABLE a (id INT); INSERT INTO a VALUES (1), (2), (3);
CREATE TABLE b (id INT); INSERT INTO b VALUES (1), (NULL);
SELECT 'NOT IN' AS method, COUNT(*) AS rows_found FROM a WHERE id NOT IN (SELECT id FROM b)
UNION ALL
SELECT 'NOT EXISTS', COUNT(*) FROM a WHERE NOT EXISTS (SELECT 1 FROM b WHERE b.id = a.id);
```

RESULT — 2 ROWS

method	rows_found
NOT IN	0
NOT EXISTS	2

Why? 2 NOT IN (1, NULL) expands to 2 <> 1 AND 2 <> NULL

TRUE AND UNKNOWN → UNKNOWN → row rejected

One NULL in the list makes every comparison UNKNOWN, so NOT IN returns nothing. NOT EXISTS has no such problem.

Figure 13. Three-valued logic strikes again.

RULE OF THUMB

Prefer `NOT EXISTS` **over** `NOT IN` for 'rows with no match', especially when the inner column is nullable. Prefer `EXISTS` over `IN` when the inner table is large; both usually plan well in MySQL 8, but only `EXISTS` is NULL-safe.

DATA-ENGINEERING LENS

Anti-joins are the heart of data validation. 'Facts with no matching dimension' (orphans), 'new rows not yet loaded', 'customers with no activity': all are `NOT EXISTS` or `LEFT JOIN … IS NULL`. Reconciliation checks between source and warehouse are exactly this query.

6.5 PART 6.5 · NAMED STEPS FOR READABLE SQL

Common Table Expressions (CTEs)

A **CTE** gives a subquery a name and puts it at the *top* of the statement with `WITH`. You then read the query top-to-bottom like a recipe: step one, step two, final answer. Same results as nested subqueries, far easier to read, test and change.

What you will learn

- Write a `WITH name AS (…)` query and chain several CTEs
- Reuse one CTE more than once in a query
- Build recursive CTEs for hierarchies and number or date series
- Pick between CTE, subquery, temporary table and view

Why it matters for data engineering. Every modern transformation framework (dbt, Dataform, SQLMesh) is built on CTE-style SQL: a chain of small named steps. Recursive CTEs also solve org-chart, bill-of-materials and calendar-generation problems that are otherwise painful.

Subquery → CTE

Take the course's nested query and unfold it. The innermost filter becomes the first CTE, the next filter the second, and the final `SELECT` reads from the last one.

SQL — The course's CTE chain, run for real

```sql
WITH cte_table AS (
    SELECT *
    FROM   dim_product
    WHERE  unit_price > (SELECT AVG(unit_price) FROM dim_product)
),
cte_table_2 AS (
    SELECT *
    FROM   cte_table
    WHERE  product_name IN ('Figure Method', 'Huge Change', 'Film Finally')
)
SELECT *
FROM   cte_table_2
WHERE  product_name = 'Figure Method';
```

RESULT — 1 ROW

product_key	product_id	product_name	category	brand	unit_price	launch_date
1	PROD0001	Figure Method	Clothing	BrandC	897.35	2020-12-06

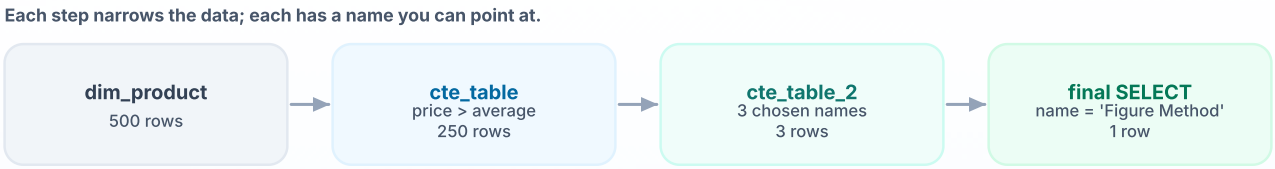


Figure 14. The CTE chain as a pipeline. Row counts are from the real run (500 → 250 → 3 → 1).

> **NOTE**
> **Syntax checklist:** the `WITH` goes *before* the main `SELECT`. Separate multiple CTEs with commas, with **no** `WITH` repeated. A later CTE may use earlier ones, never later ones. A CTE exists only for that one statement; it is not stored.

Reusing a CTE twice

A big advantage over subqueries: you define the logic once and reference it as often as you like. Here we compare each month of 2024 with the same month of 2025 using a self-join on a CTE.

SQL — Year-over-year, Jan to Oct (the data ends mid-October 2025, so the last month is partial)

```sql
WITH m AS (
    SELECT LEFT(ym, 4) AS yr, RIGHT(ym, 2) AS mm, revenue FROM monthly_rev
)
SELECT a.mm AS month, a.revenue AS rev_2024, b.revenue AS rev_2025,
       ROUND(100 * (b.revenue - a.revenue) / a.revenue, 1) AS yoy_pct
FROM   m a
JOIN   m b ON b.mm = a.mm AND a.yr = '2024' AND b.yr = '2025'
ORDER  BY a.mm;
```

RESULT — 10 ROWS

month	rev_2024	rev_2025	yoy_pct
01	690291	582157	-15.7
02	618678	483032	-21.9
03	590172	509061	-13.7
04	613941	565819	-7.8
05	588144	585742	-0.4
06	589776	609098	3.3
07	583645	555998	-4.7
08	604382	626050	3.6
09	473587	572634	20.9
10	583660	200506	-65.6

Recursive CTEs

A recursive CTE refers to **itself**. It has two parts joined by `UNION ALL`: an **anchor** that seeds the first rows, and a **recursive member** that builds the next layer from the previous one. It stops when a round produces no new rows.

SQL — The shape of every recursive CTE

```sql
WITH RECURSIVE name AS (
    SELECT …                -- anchor: runs once
    UNION ALL
    SELECT … FROM name …    -- recursive member: runs again and again
)
SELECT * FROM name;
```

Example A · Number series

SQL — The `WHERE n < 8` is the stop condition. Without it the recursion never ends.

```sql
WITH RECURSIVE n AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM n WHERE n < 8
)
SELECT n, n * n AS square FROM n;
```

RESULT — 8 ROWS

n	square
1	1
2	4
3	9
4	16
5	25
6	36
7	49
8	64

⚠ WATCH OUT

MySQL 8 stops runaway recursion at `cte_max_recursion_depth` (default **1000**) with an error. Always include a terminating condition. (MariaDB uses `max_recursive_iterations`.)

Example B · Org chart traversal

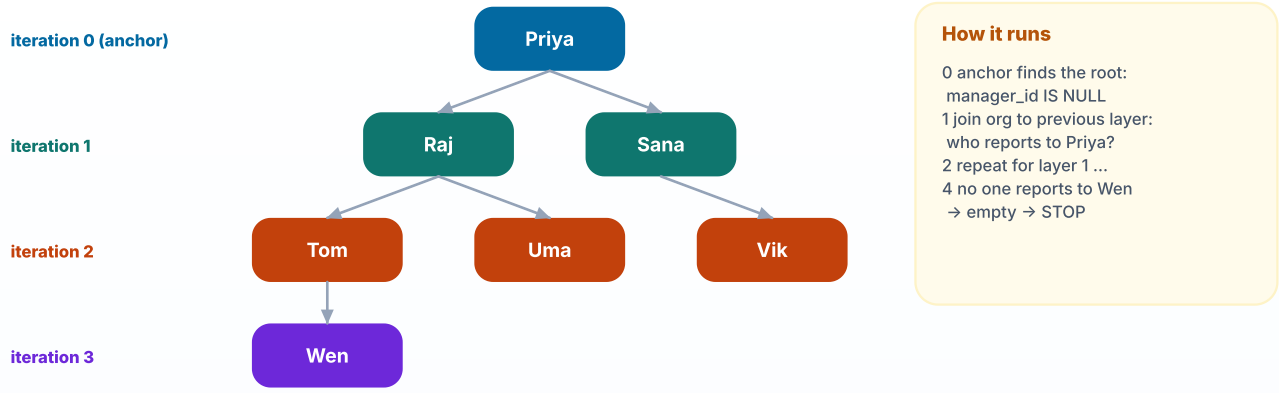


Figure 15. Each iteration joins the base table to the rows produced by the iteration before it.

SQL — Every employee with their depth and full reporting path

```sql
WITH RECURSIVE chain AS (
    SELECT id, name, manager_id, 0 AS depth, CAST(name AS CHAR(100)) AS path
    FROM   org
    WHERE  manager_id IS NULL
    UNION ALL
    SELECT o.id, o.name, o.manager_id, c.depth + 1, CONCAT(c.path, ' > ', o.name)
    FROM   org o
    JOIN   chain c ON o.manager_id = c.id
)
SELECT depth, path FROM chain ORDER BY path;
```

RESULT — 7 ROWS

depth	path
0	Priya
1	Priya > Raj
2	Priya > Raj > Tom
3	Priya > Raj > Tom > Wen
2	Priya > Raj > Uma
1	Priya > Sana
2	Priya > Sana > Vik

DATA-ENGINEERING LENS

Hierarchies live in warehouses too: product category trees, GL account roll-ups, bill-of-materials, referral chains. A parent-id column plus a recursive CTE is the standard way to flatten them for BI tools.

Example C · Filling gaps in a calendar

SQL — `daily` has no Jan 5 or Jan 6. The generated calendar plus LEFT JOIN makes the gaps visible as zeros.

```sql
WITH RECURSIVE cal AS (
    SELECT DATE('2025-01-01') AS d
    UNION ALL
    SELECT d + INTERVAL 1 DAY FROM cal WHERE d < '2025-01-06'
)
SELECT cal.d, COALESCE(SUM(daily.amount), 0) AS amount
FROM   cal
LEFT   JOIN daily ON daily.day = cal.d
GROUP  BY cal.d;
```

RESULT — 6 ROWS

d	amount
2025-01-01	100
2025-01-02	250
2025-01-03	150
2025-01-04	300
2025-01-05	0
2025-01-06	0

CTE, subquery, temp table or view?

	Subquery	CTE	Temporary table	View
Lifetime	inside one clause	one statement	your session	permanent definition
Reusable in the same query	no, copy-paste	**yes**, by name	yes	yes
Stores data?	no	no	**yes**	no (re-runs the query)
Can be indexed	no	no	**yes**	no
Recursion	no	**yes**	no	no
Best for	one-off filters	readable multi-step logic	heavy intermediate results reused many times	sharing logic across queries and users

INTERVIEW ANGLE

"Is a CTE faster than a subquery?" Usually **no**: the optimizer treats them alike (MySQL 8 may merge or materialize either). A CTE buys *readability and reuse*, not speed. If an intermediate result is huge and used repeatedly, a temp table with an index can win.

6.6 WRAP-UP

Phase 6 Recap & Practice

A one-page summary of the analytics toolbox, then five exercises that combine windows, subqueries and CTEs.

Cheat sheet

Need	Pattern
Group total beside each row	`SUM(x) OVER (PARTITION BY g)`
Running total	`SUM(x) OVER (ORDER BY d ROWS UNBOUNDED PRECEDING)`
Moving average (3 rows)	`AVG(x) OVER (ORDER BY d ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`
Previous / next value	`LAG(x, 1, 0) OVER (ORDER BY d)` · `LEAD(...)`
Last value of a group	`LAST_VALUE(x) OVER (… ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)`
Unique numbering / ties share / ties no gap	`ROW_NUMBER()` / `RANK()` / `DENSE_RANK()`
Buckets and percentiles	`NTILE(4)` · `PERCENT_RANK()` · `CUME_DIST()`
Top-N per group	rank in a subquery or CTE, then `WHERE rn <= N` outside
Compare to an overall value	`WHERE x > (SELECT AVG(x) FROM t)`
Has / has not a related row	`EXISTS` / `NOT EXISTS` (not `NOT IN` with NULLs)
Multi-step logic	`WITH a AS (…), b AS (SELECT … FROM a) SELECT … FROM b`
Hierarchy or series	`WITH RECURSIVE x AS (anchor UNION ALL step) …`

Check yourself

Q1 How is a window function different from `GROUP BY`?

It computes over a set of rows but returns **one output row per input row**; `GROUP BY` collapses rows into one per group.

Q2 Why can you not write a window function in `WHERE`?

Windows are evaluated after `WHERE` / `GROUP BY` / `HAVING`. Put the window in a subquery or CTE and filter in the outer query.

Q3 Difference between `RANK` and `DENSE_RANK`?

Both give tied rows the same number; `RANK` then skips numbers (1, 2, 2, 4), `DENSE_RANK` does not (1, 2, 2, 3).

Q4 Why does `LAST_VALUE` often return the current row?

The default frame ends at `CURRENT ROW`. Use `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.

Q5 What is a correlated subquery?

An inner query that references a column of the outer row, so it is evaluated once per outer row (conceptually).

Q6 Why prefer `NOT EXISTS` to `NOT IN`?

If the subquery returns any NULL, `NOT IN` yields UNKNOWN for every row and returns nothing.

Q7 What are the two parts of a recursive CTE?

The **anchor** (seed rows) and the **recursive member** (uses the CTE itself), joined by `UNION ALL`, with a stop condition.

Practice lab

1. For the first 5 sales (`sales_id` order), show `region`, `total_amount` and the **average total_amount of that region** next to it (window function).
2. Find the **top 3 customers by total spend**. Show rank, full name and total. Use CTEs and `RANK`.
3. List every month whose revenue is **above the average month** (use `monthly_rev` and a scalar subquery).
4. With a recursive CTE, generate the numbers 1 to 5 and their cubes.
5. Show the **cheapest product in each category** using `ROW_NUMBER` ascending.

Answer key

SQL 1

```sql
SELECT   f.sales_id, s.region, f.total_amount,
         ROUND(AVG(f.total_amount) OVER (PARTITION BY s.region), 2) AS region_avg
FROM     fact_sales f JOIN dim_store s ON s.store_key = f.store_key
ORDER BY f.sales_id
LIMIT    5;
```

RESULT 5 ROWS

sales_id	region	total_amount	region_avg
1	Central	94.04	2795.71
2	North	1324.55	2721.66
3	South	1004.53	2861.77
4	West	5949.11	2812.50
5	Central	360.13	2795.71

SQL 2

```sql
WITH spend AS (
    SELECT customer_key, ROUND(SUM(total_amount)) AS total FROM fact_sales GROUP BY customer_key
), ranked AS (
    SELECT customer_key, total, RANK() OVER (ORDER BY total DESC) AS rnk FROM spend
)
SELECT r.rnk, CONCAT(c.first_name, ' ', c.last_name) AS customer, r.total
FROM   ranked r JOIN dim_customer c ON c.customer_key = r.customer_key
WHERE  r.rnk <= 3
ORDER  BY r.rnk;
```

RESULT 3 ROWS

rnk	customer	total
1	Timothy York	66869
2	Todd Lucas	64409
3	Sarah Lee	61737

SQL 3

```sql
SELECT ym, revenue FROM monthly_rev WHERE revenue > (SELECT AVG(revenue) FROM monthly_rev) ORDER BY ym;
```

RESULT 18 ROWS

ym	revenue
2023-11	619450
2023-12	620670
2024-01	690291
2024-02	618678
2024-03	590172
2024-04	613941

... 12 more rows not shown

SQL 4

```sql
WITH RECURSIVE n AS (SELECT 1 AS n UNION ALL SELECT n + 1 FROM n WHERE n < 5)
SELECT n, POWER(n, 3) AS cube FROM n;
```

RESULT 5 ROWS

n	cube
1	1
2	8
3	27
4	64
5	125

SQL 5

```sql
SELECT category, product_name, unit_price
FROM (
    SELECT category, product_name, unit_price,
           ROW_NUMBER() OVER (PARTITION BY category ORDER BY unit_price) AS rn
    FROM   dim_product
) t
WHERE rn = 1
ORDER BY category;
```

RESULT 6 ROWS

category	product_name	unit_price
Books	Actually Southern	16.75
Clothing	My Hard	14.24
Electronics	Environment Performance	12.22
Home & Kitchen	Look Interest	12.15
Sports	Middle Forward	11.82
Toys	Sea Actually	14.64

COMING IN PHASE 7

Phase 7: Production SQL. Real-time scenarios (nth value, de-duplication, LAG/LEAD), views, stored procedures, user-defined functions, and a roadmap of what to learn next for data engineering.